



Carnegie Mellon University

Master of
Software Engineering

Security – authentication and authorization

LG Architecture Training Program

Paulo Merson

Agenda



➔ Security tactics
Security in the SDLC

Security tactics

- Verify message integrity
- Limit exposure
- Encrypt data
- IP restriction
- Validate input
- Audit
- Authenticate actors
- Authorize actors



Authenticate actors

- Tactic that helps to **resist** attacks
- Authentication is the process to verify that an actor is who or what it claims to be
- Actors are users, systems, and other entities that need access to a computational resource (e.g., a web service)
- Actors must have an identity (user ID, login, email address, SSH key, API key, etc.)

*Subject is another term
for actor or user in
access management
solutions*



Multi-factor authentication

- For the authentication of end users, we can employ multi-factor authentication (MFA) or specifically two-factor authentication (2FA)
- It helps to prevent identity theft
- MFA consists of requiring 2+ authentication methods (of different categories):



**Information only
the user knows**

Examples:

- Password
- Security question



**Something only the
user has**

Examples:

- Phone
- USB token



**Something only the
user "is"**

Examples:

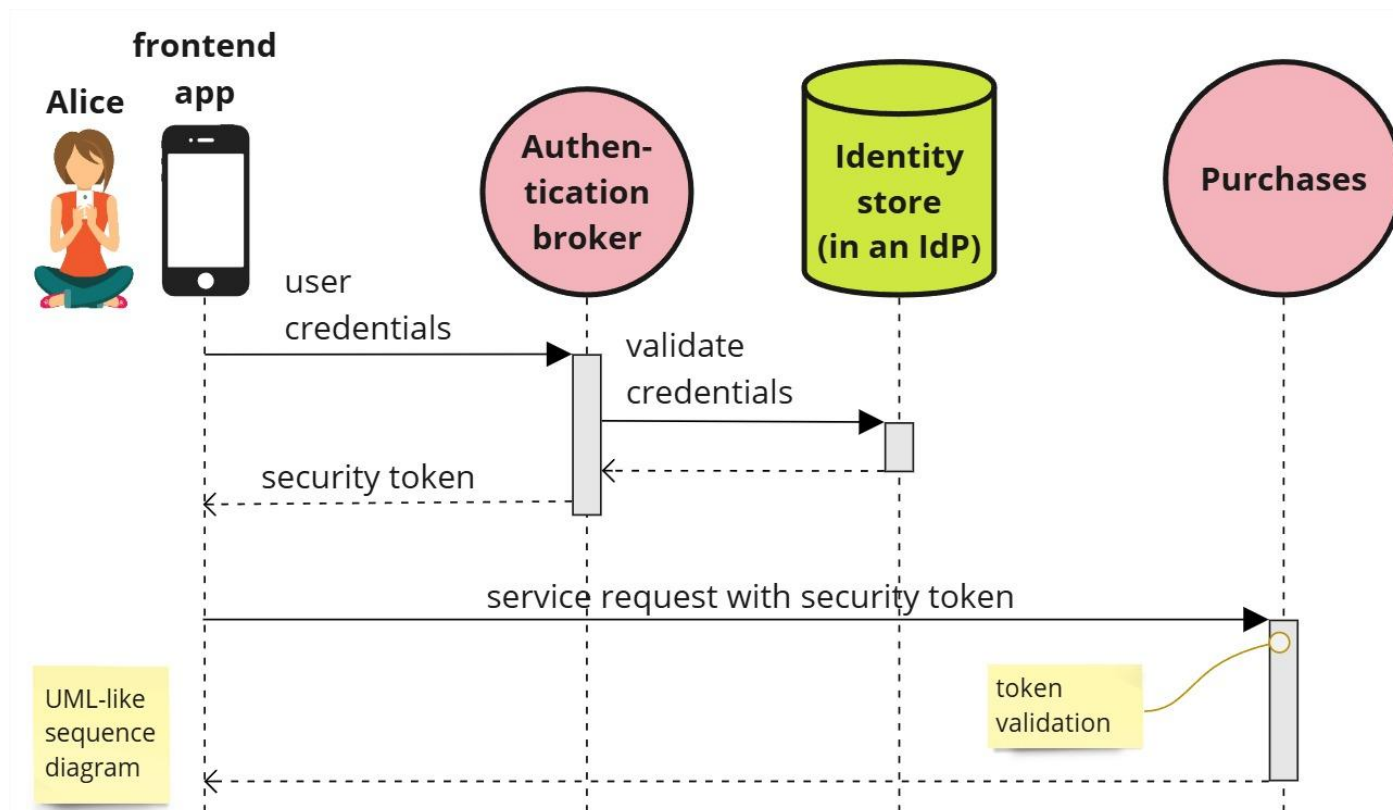
- Fingerprint
- Face recognition

Identity provider (IdP)

- Authentication requires an identity provider, which is a software solution that
 - stores and manages data needed for access control in a security scope (security domain)
 - provides authentication services
- It may integrate with other security domains to allow single sign-on (SSO)
- The overall authentication solution involves many services, such as
 - Provisioning and deprovisioning of accounts
 - Self-registration (optional)
 - Password reset
 - MFA
 - Session or connection timeout

Brokered authentication pattern

- The client application sends user credentials to an authentication broker that validates the credentials and responds with a security token
- The application then uses the token to make calls to services



Security token is aka access token

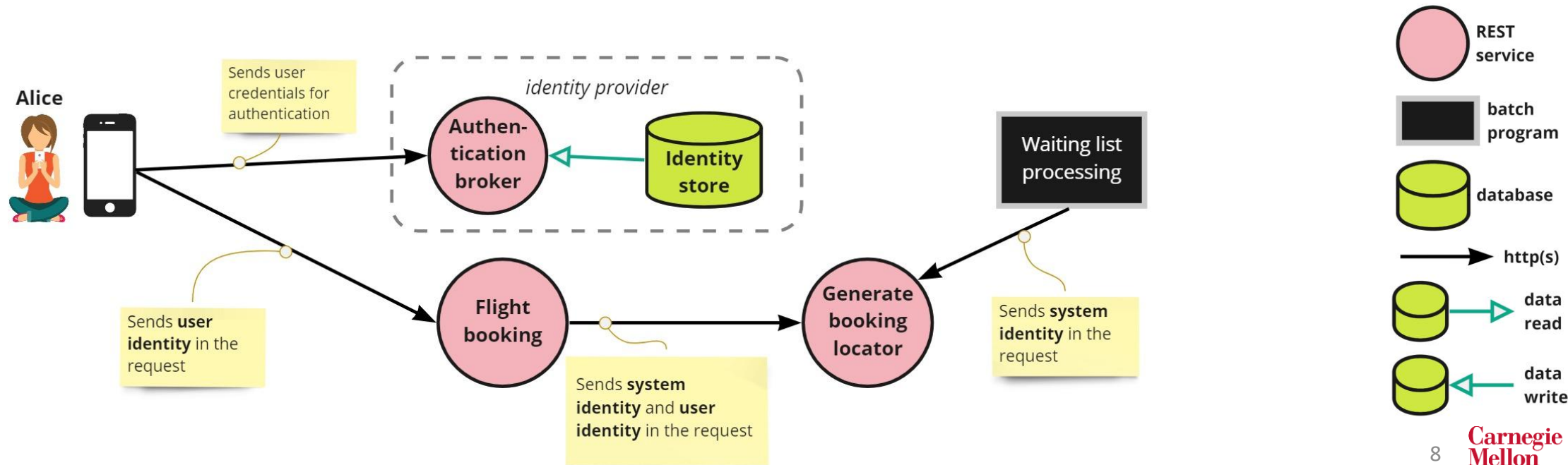
Token validation may involve calling the broker

Client uses the same token until it expires

can also be used for authorization

User identity and system identity

- When a component is called by another component, instead of a *user identity* the called component may require a *system identity*
 - **User identity** unequivocally identifies a human actor
 - **System identity** identifies a software system, service, or software component in general, and is used in machine-to-machine (M2M) calls





Authorize actors

- Tactic that helps to **resist** attacks
- Authorization is the process of verifying that an authenticated actor has the rights to perform an action
 - The action can be reading or writing data, executing a component operation, etc.
- Identity providers also manage granting and revoking rights
- These rights can be granted to users, groups of users, or roles



Why use
groups or
roles?

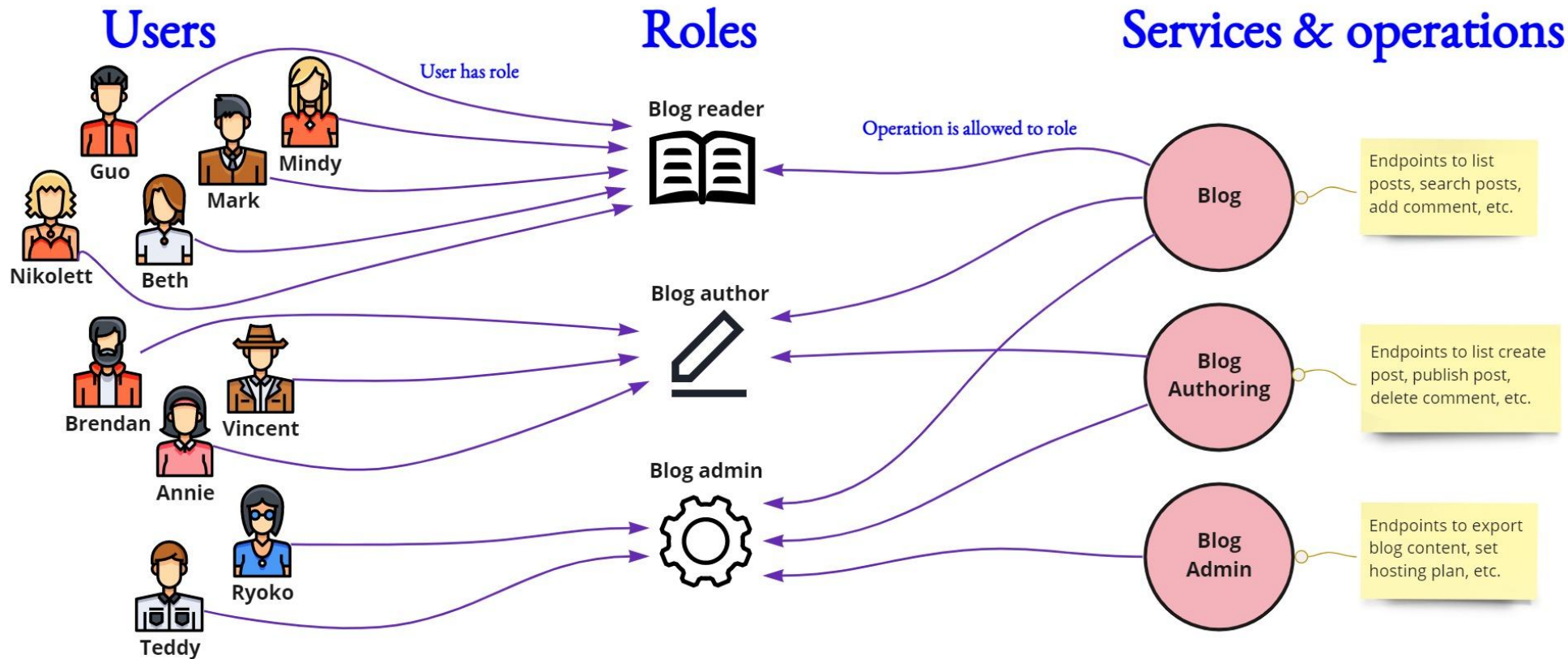
*Individually assigned authorization rights
create a provisioning and deprovisioning
burden that may result in security loopholes*

Role-Based Access Control (RBAC)

- Data access or operation execution rights specify what roles are allowed
 - In other words, rights are granted to roles not actors
- An actor may have 1+ roles
 - A system may restrict the number of active roles in a session
- Rules establish what actors may have a given role
 - Assigning and removing roles to and from a user should ideally be automated

RBAC example

- A given operation may be allowed to multiple roles (as in the diagram)



Alternative: roles may form a role hierarchy

Ex.: Blog admin inherits rights of Blog author, which inherits rights from Blog reader

Principle of Least Privilege (PoLP)

- An actor should have only the minimum rights necessary to perform a given action
- When referring to access to *information*, this principle is dubbed “need-to-know basis”
- This principle should guide the granting of access rights

When this principle is not followed and actors have rights they don't need, the attack surface increases and the system is more vulnerable



*GenAI can help with
challenges not
addressed by tactics
in this course!*

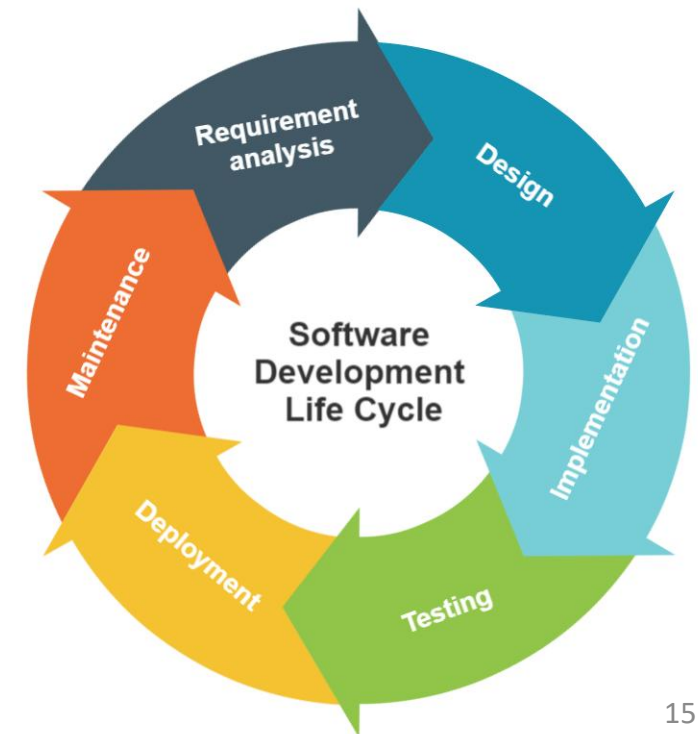
Agenda

Security tactics

➔ Security in the SDLC

Security outside the software architecture

- We have focused on **designing software with security**
- But security is a quality attribute that pervades all phases of software development



Awareness of application security

- Developers should be aware of common vulnerabilities and security best practices
- OWASP.org is a well-known, trusted source of information (“OWASP Top 10 Web Application Security Risks”, other top 10s, guidelines, etc.)
- Effective application security requires **3 measures**:



Education

Make the team aware of security risks and development best practices

Should be part of new developers' onboarding



Policies

Define in written the policies and constraints that teams should follow

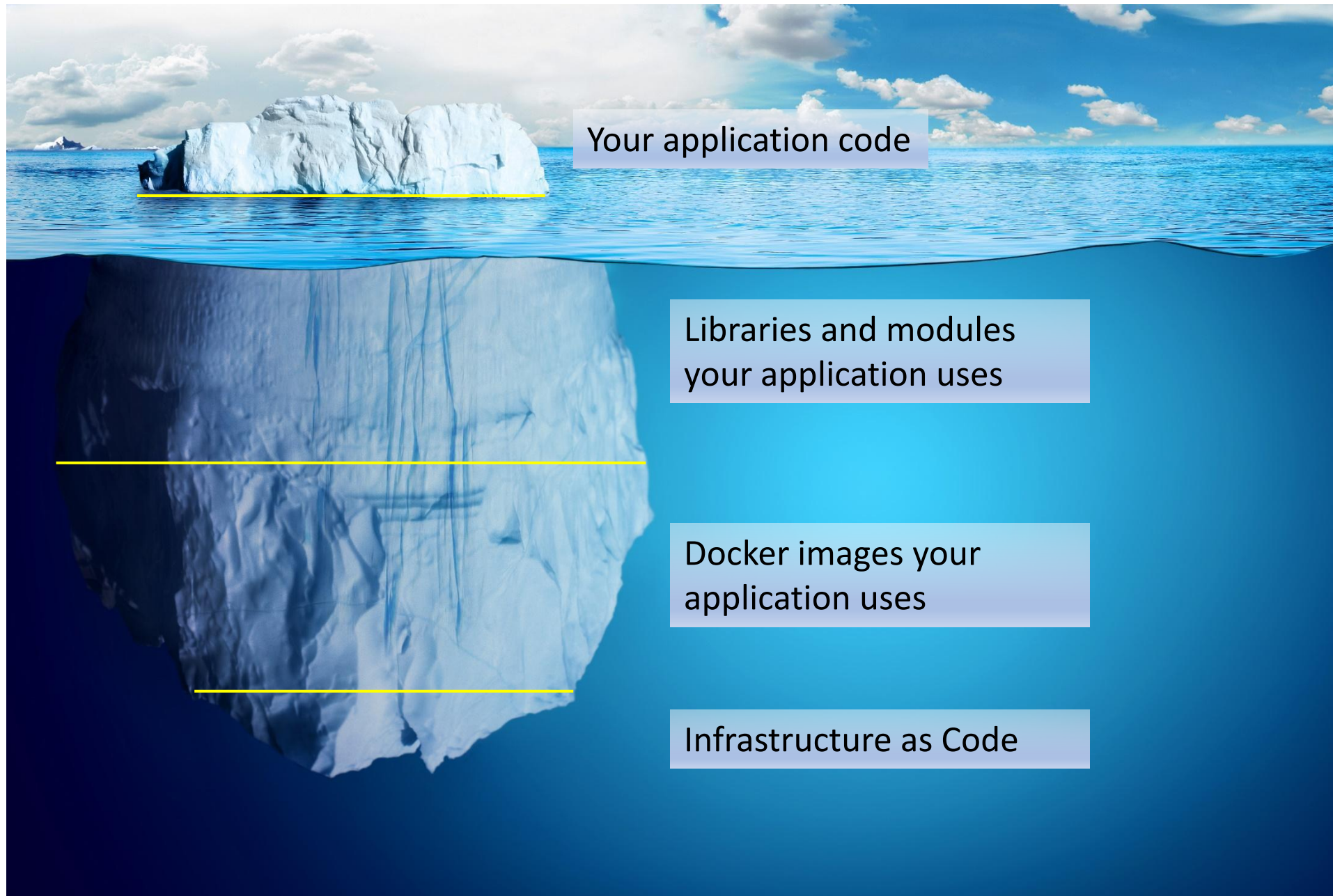
Should define accountability



Enforcement

Create mechanisms to check conformance with security policies

Automation is key (next 2 slides)



Tools to detect vulnerabilities *in application artifacts*

Type of tool	What it does	Examples of tools
Static Application Security Testing (SAST)	Parse and analyze source code to identify known vulnerabilities	Checkmarx, Coverity, Fortify, Klocwork, Mend SAST, Sonarqube, Snyk Code, Veracode, Semgrep, HCL AppScan,
Dynamic Application Security Testing (DAST)	Test services at runtime simulating an attack and checking the result	AppCheck, Burp, Detectify, HCL AppScan, Indusface WAS, Invicti, StackHawk, ZAP, Veracode, Fortify WebInspect
Dependency scanner	Analyze libraries and modules included in the application project to find known vulnerabilities through dependency paths	Debricked, Dependabot, Jfrog Xray, Mend SCA, OWASP Dependency-Check/-Track, Snyk Open Source, Trivy, Grype
Container image scanner	Scan the (layered) docker images used in your application to identify known vulnerabilities	Anchore, Aqua Trivy, Clair, Jfrog Xray, Quay, Snyk Container, Grype, Docker Sout
IaC security tool	Parse your IaC files and point out known vulnerabilities and your own rules/policies	Aqua, CloudSploit, Checkov, Kyverno, Kubesec, OPA, Snyk IaC, Sysdig, Terrascan, Terraform Sentinel

Important particularly for distributed systems



Questions?

Paulo Merson
pmerson@cmu.edu